StrangerTEC Morse Translator

Esteban Alejandro Sánchez Ledezma y Gerald Andrés Calderon Navarro

Tecnológico de Costa Rica

Documento Técnico

Leonardo Andres Araya Martinez

2026

Introducción

El presente proyecto consiste en el desarrollo de un sistema interactivo basado en la comunicación mediante código Morse, inspirado en la serie *Stranger Things*. La solución implementa una maqueta física con un panel de luces y un sistema de entrada mediante pulsadores, controlado por una Raspberry Pi Pico W, junto con una interfaz gráfica desarrollada en Python.

El objetivo principal es permitir la comunicación entre dos usuarios a través de señales luminosas y sonoras, simulando el envío y recepción de mensajes en código Morse. Además, el sistema incorpora una dinámica de juego en la cual se evalúa la precisión y rapidez de los participantes al interpretar y transmitir mensajes.

En este documento se encontrará a gran medida la información sobre el desarrollo del proyecto, puntos clave como conclusiones, análisis de resultados y fuentes consultadas son algunos de los puntos clave del documento.

Índice

Conclusiones 4
Recomendaciones 5
Análisis de Resultados 6
Bibliotecas utilizadas 7
Diagramas 8
Referencias y Fuentes Consultadas 11
Diagramas de Flujo, Diseño Lógico y Planificación 11
Raspberry Pi Pico W / MicroPython / Thonny 11
Instalación, configuración y uso: 12
Gestión del Proyecto y Control de Versiones 14
Referencias y consultas con la IA 15

Conclusiones

El desarrollo del proyecto permitió aplicar de manera práctica conceptos relacionados con sistemas embebidos, electrónica digital y programación en Python.

Se logró implementar un sistema funcional capaz de interpretar y transmitir mensajes en código Morse, utilizando tanto medios físicos como digitales.

Además, se evidenció la importancia de la correcta planificación y división del trabajo para integrar adecuadamente los distintos componentes del sistema.

Finalmente, el proyecto demostró cómo tecnologías simples pueden ser utilizadas para crear sistemas interactivos y educativos.

Recomendaciones

Se recomienda realizar pruebas individuales de cada componente antes de la integración total del sistema, con el fin de facilitar la detección de errores.

Además, es importante calibrar adecuadamente los tiempos de los pulsos para garantizar una correcta interpretación del código Morse.

Se sugiere también mejorar la interfaz gráfica para hacerla más intuitiva y amigable para el usuario.

Finalmente, para futuras versiones del proyecto, se podría implementar una comunicación inalámbrica más robusta y ampliar el conjunto de funcionalidades del sistema.

Análisis de Resultados

Durante el desarrollo del proyecto, se evaluó el funcionamiento del sistema en términos de precisión en la detección del código Morse, respuesta del hardware y correcta comunicación entre los componentes. Se observó que la correcta temporización de los pulsos (puntos y rayas) es fundamental para garantizar una interpretación adecuada de los mensajes. Asimismo, la retroalimentación mediante luces y sonido permitió mejorar la interacción del usuario con el sistema. El sistema logró cumplir con los objetivos planteados, permitiendo la transmisión y recepción de mensajes, así como la evaluación del desempeño de los jugadores mediante un sistema de puntuación.

Bibliotecas utilizadas

- tkinter: Biblioteca empleada para la interfaz gráfica
- random: Biblioteca necesaria para crear valores aleatoria (decisiones del CPU)
- time: Utilizada para crear contadores
- Toplevel (de tkinter): Función de tkinter que permite priorizar widgets, utilizado para la permanencia del fondo en la interfaz gráfica
- Pin PWM (de machine): Función propia de Python utilizada para el manejo de los pines (leds) y del buzzer.
- sys: Biblioteca utilizada para el envío de mensajes codificados
- serial: Biblioteca utilizada para obtener información sobre los puertos de la computadora.
- serial.tools.list_ports : Función derivada de la biblioteca serial, implementa diferentes herramientas como escaneo de puertos activos o puertos usados.
- Threading: Biblioteca que permite a Python distribuir funciones y organizarlas para ejecutarlas en paralelo. Utilizada para poder ejecutar la comunicación con la maqueta sin comprometer el código de la interfaz
- Network: Biblioteca que implementa múltiples herramientas de enrutamiento, transmisión de datos y comunicación entre dispositivos. Utilizado para la comunicación inalámbrica de la maqueta.

Diagramas

Figura 1. Diagrama de arquitectura básico

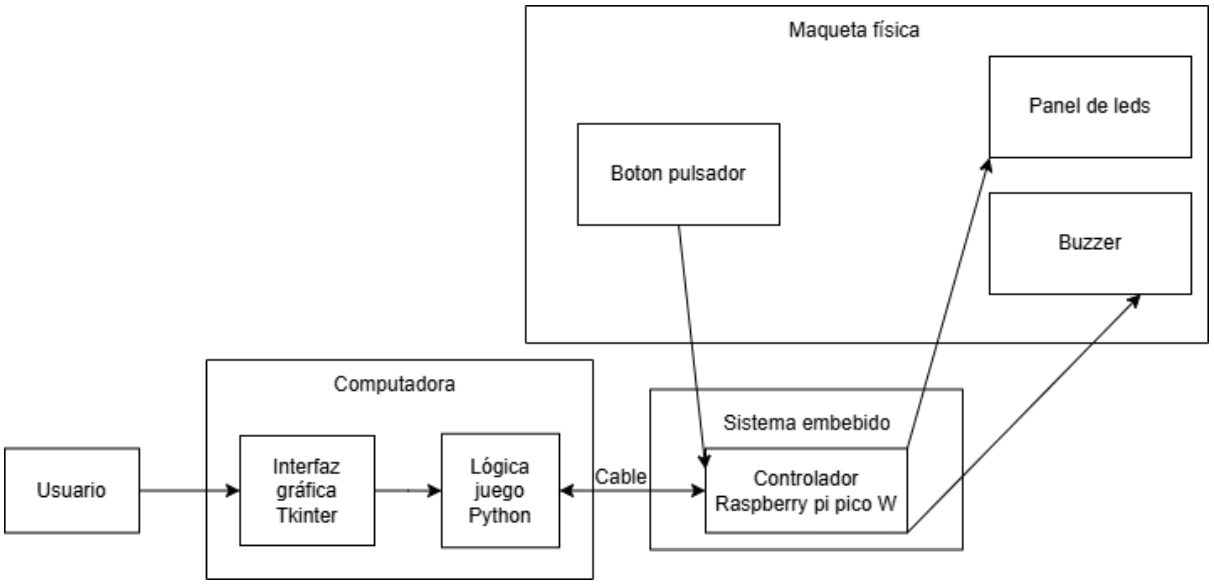


Figura 2. Diagrama de módulos del sistema

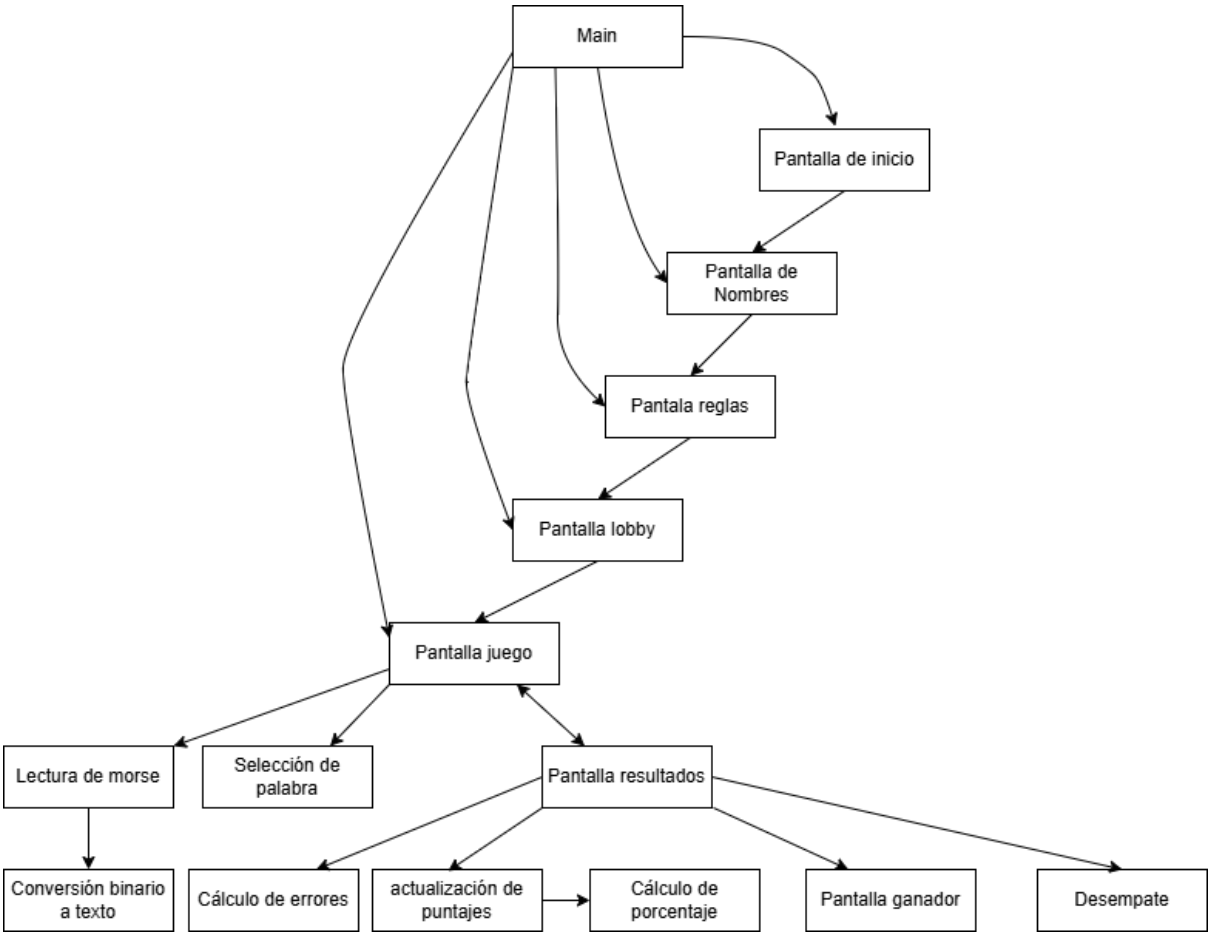
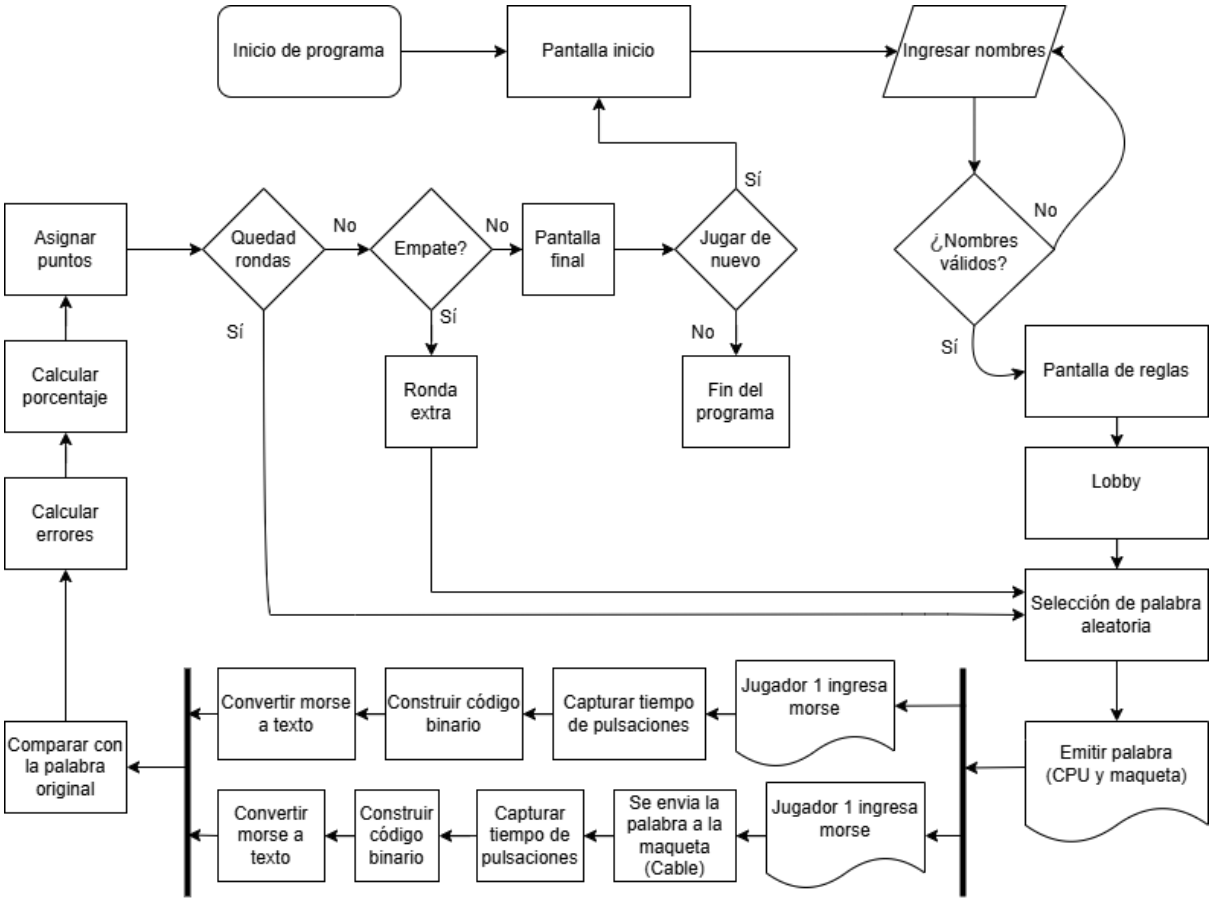


Figura 3. Diagrama de flujo:



Referencias y Fuentes Consultadas

Desarrollo de Software e Interfaz Gráfica (Python / Tkinter)

Lista de reproducción sobre el uso de Tkinter para el desarrollo de interfaces gráficas en Python (español):

https://www.youtube.com/playlist?list=PL7HAy5R0ehQXb2aFKOKyeCMequxyb5jzJ

Python Tkinter GUI Tutorial en Español – Curso práctico para interfaces:

https://www.youtube.com/watch?v=h8usxRykVmI&list=PLVzwufPir357GruwLwGXCi_PXrbsASqEv

Documentación oficial de Python:

https://docs.python.org/3/

Documentación oficial de Tkinter:

https://docs.python.org/es/3/library/tkinter.html

Diagramas de Flujo, Diseño Lógico y Planificación

Video sobre diseño de diagramas de flujo:

https://www.youtube.com/watch?v=zNmTsj8J89g&t=901s

Lucidchart (herramienta de diagramas):

https://www.lucidchart.com/

Draw.io / Diagrams.net (diagramas gratuitos):

https://app.diagrams.net/

Raspberry Pi Pico W / MicroPython / Thonny

Instalación, configuración y uso:

- Página oficial Raspberry Pi Pico:

  https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico/

- Documentación oficial MicroPython:

  https://micropython.org/

- Instalación de Thonny IDE:

  https://thonny.org/

Tutorial Raspberry Pi Pico + Micro Python + Thonny (español):

youtube.com/watch?v=r9rj9qFWHNc&pp=ygUhcmFzcGJlcnJ5IHBpIHBpY28gdGhvbm55IGVzcGHDsW9s0gcJCd4KAYcqIYzv

https://projects-raspberrypi-org.translate.goog/en/projects/get-started-pico-w?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

Curso Raspberry Pi Pico W con Micro Python (español):

https://www.youtube.com/watch?v=2cqeFDw0VLc&list=PLF-qcfymUY4WOMNac4TKCMsccixB91K68

PySerial Documentation:

https://pyserial.readthedocs.io/

Raspberry Pi Pico W Wireless Guide:

https://projects.raspberrypi.org/en/projects/get-started-pico-w

Tutorial sockets MicroPython:

https://www.youtube.com/watch?v=M1MltJsdmcM

Redes cliente-servidor en MicroPython:

https://www.youtube.com/watch?v=RonINjKL-I0

Datasheet SN74LS164N:

https://www.alldatasheet.com/datasheet-pdf/view/12629/ONSEMI/SN74LS164N.html

Explicación práctica del 74HC164:

https://www.youtube.com/watch?v=RHaBIaxr2IM

Datasheet L7805CV:

https://www.alldatasheet.com/datasheet-pdf/view/22634/STMICROELECTRONICS/L7805CV.html

Polaridad, resistencias y uso práctico:

https://www.youtube.com/watch?v=PZOvLbRdgNM&pp=ygUfbGVkIHJlc2lzdGVuY2lhIHB5dGhvbiBlc3Bhw7FvbA%3D%3D

Uso de botones con Raspberry Pi Pico:

https://www.youtube.com/watch?v=d_odoKbEjgg

Uso de buzzer con Raspberry Pi Pico:

https://www.youtube.com/watch?v=A3hOZvgyBH0

Historia y alfabeto Morse:

https://morsecw.com/alfabeto.html#espaciado

Espaciado y temporización Morse:

https://morsecw.com/espaciado.gif

Referencia internacional del código Morse:

https://es.wikipedia.org/wiki/C%C3%B3digo_morse

Wokwi Simulator (simulación de Raspberry Pi Pico):

https://wokwi.com/

Tinkercad Circuits (simulación de electrónica):

https://www.tinkercad.com/

Gestión del Proyecto y Control de Versiones

GitHub:

https://github.com/

GitHub Docs:

https://docs.github.com/

Guía básica Git y GitHub (español):

https://www.youtube.com/watch?v=44ziZ12rJwU&t=53s&pp=ygUWZ2l0aHViIGJhc2ljbyBlc3Bhw7FvbNIHCQneCgGHKiGM7w%3D%3D

Referencias y consultas con la IA

Se le consultó a la IA diferentes problemas, pero se le comento antes que no debia responder directamente sino que dar pistas o construir un camino para poder llegar a la solución estos son las principales consultas y las más importantes:

Chat 1 — Threading

Yo: Tengo una interfaz en tkinter que se congela cuando espera datos de un puerto serial, ¿cómo puedo evitar que se congele?

IA: El problema es que tkinter corre en un solo hilo y cuando bloqueas ese hilo esperando datos, la interfaz deja de responder. La solución es correr la operación que espera en un hilo separado usando threading. Te dejo la idea general, la función que espera datos la ponés dentro de un Thread y cuando termina le avisás al hilo principal. Investigá threading.Thread y cómo comunicar resultados entre hilos sin tocar tkinter directamente desde el secundario.

Chat 2 — Puerto serial bloqueado

Yo: Tengo una Raspberry Pi Pico conectada por USB y cuando intento conectarme con pyserial me dice que no encuentra el puerto, pero sé que está conectada porque Thonny la detecta.

IA: El problema es que Thonny mantiene una conexión serial abierta con la Pico para poder ejecutar código. Dos programas no pueden usar el mismo puerto al mismo tiempo. Tenés que cerrar Thonny antes de que tu programa intente conectarse. También podés buscar el puerto en Thonny antes de cerrarlo, aparece en la esquina inferior derecha, y usarlo directamente en tu código.

Chat 3 — Widgets destruidos en tkinter

Yo: Tengo una función recursiva con after() que cambia el color de un label para hacer un efecto de parpadeo, pero cuando cambio de pantalla me tira un error diciendo invalid command name.

IA: Lo que pasa es que after() programa una llamada futura y aunque destruyas el widget, esa llamada igual se ejecuta después. Hay dos formas de resolverlo: guardás el ID que devuelve after() y lo cancelás con after_cancel() antes de cambiar de pantalla, o dentro de la función verificás si el widget sigue existiendo con winfo_exists() antes de tocarlo. La segunda es más simple para tu caso.

Chat 4 — Registros de corrimiento

Yo: Tengo dos registros de corrimiento 74LS164 en cascada y necesito controlar 13 LEDs con solo 2 pines de la Raspberry Pi Pico, cómo funciona eso?

IA: El 74LS164 es un registro de desplazamiento serial a paralelo, recibe bits uno por uno y los va desplazando hacia sus 8 salidas. Con dos en cascada tenés 16 salidas controladas por 2 pines: uno de datos y uno de clock. Para encender el LED número N mandás 16 bits donde solo el bit N vale 1 y los demás 0, dando un pulso de clock por cada bit. Investigá cómo funciona el desplazamiento de bits y cómo conectar la salida QH del primer registro a la entrada del segundo.

Chat 5 — Protocolo de comunicación serial

Yo: La Raspberry Pi Pico me manda mensajes de confirmación por el puerto serial pero mi programa en la PC los confunde con los datos reales que necesito procesar, cómo los distingo?

IA: Necesitás un protocolo simple para separar los tipos de mensajes. Una forma común es usar un prefijo especial para los mensajes de debug, por ejemplo que todos los mensajes informativos empiecen con # y los datos reales no lleven prefijo. En la PC filtrás leyendo línea por línea e ignorando las que empiecen con #. Investigá cómo leer línea por línea con readline() en pyserial y cómo filtrar strings en Python con startswith().